

Course 8457 - Weeks 1, 2, 3 Quiz Compilation

1. **Which model executes code line-by-line?**
 - ☐ Static
 - ☐ Compiled
 - ☐ Hybrid
 - ☒ Interpreted
2. **Which phase generates machine-level instructions?**
 - ☒ Code generation
 - ☐ Parsing
 - ☐ Lexical analysis
 - ☐ Optimization
3. **Which language is a classic example of procedural programming?**
 - ☐ Haskell
 - ☒ C
 - ☐ Prolog
 - ☐ Scala
4. **Interpretation occurs _____ execution.**
 - ☐ After
 - ☐ Before
 - ☒ During
 - ☐ Never
5. **Which paradigm separates logic from control flow?**
 - ☐ Imperative
 - ☒ Declarative
 - ☐ Procedural
 - ☐ Object-Oriented
6. **Which paradigm organizes code around objects and classes?**
 - ☐ Functional
 - ☒ Object-Oriented
 - ☐ Procedural
 - ☐ Declarative
7. **Which model combines compilation and interpretation?**
 - ☐ Compiled
 - ☒ Hybrid
 - ☐ Interpreted
 - ☐ Manual
8. **Which compiler phase converts source code into tokens?**
 - ☒ Lexical analysis
 - ☐ Parsing
 - ☐ Optimization
 - ☐ Code generation

9. *Hybrid models use both compiler and _____.*
- ☐ Debugger
 - ☐ Assembler
 - ☐ Loader
 - ☒ **Interpreter**
10. *Which paradigm emphasizes immutability and pure functions?*
- ☒ **Functional**
 - ☐ Procedural
 - ☐ Object-Oriented
 - ☐ Logic
11. *Semantics defines the _____ of constructs.*
- ☐ Symbols
 - ☐ Structure
 - ☒ **Meaning**
 - ☐ Syntax
12. *Compilation occurs _____ program execution.*
- ☒ **Before**
 - ☐ During
 - ☐ After
 - ☐ Never
13. *Which language is strongly associated with functional programming?*
- ☐ C
 - ☐ Java
 - ☒ **Haskell**
 - ☐ Pascal
14. *Syntax refers to the _____ of a language.*
- ☐ Meaning
 - ☒ **Structure**
 - ☐ Usage
 - ☐ Execution
15. *Modern languages support multiple paradigms.*
- ☐ Rarely
 - ☐ False
 - ☒ **True**
 - ☐ Never
16. *Object-oriented programming relies on _____ for structure.*
- ☒ **Classes**
 - ☐ Registers
 - ☐ Instructions
 - ☐ Scripts

17. *Lexical analysis identifies _____.*
- ☐ Trees
 - ☒ **Tokens**
 - ☐ Instructions
 - ☐ Files
18. *Assembly language belongs to the _____ generation.*
- ☐ Fourth
 - ☐ First
 - ☐ Third
 - ☒ **Second**
19. *Which paradigm is closest to hardware execution style?*
- ☒ **Imperative**
 - ☐ Functional
 - ☐ Logic
 - ☐ Declarative
20. *Code generation produces _____ code.*
- ☐ Token
 - ☐ Source
 - ☒ **Machine**
 - ☐ Tree
21. *Compiled languages typically offer _____ performance.*
- ☐ Moderate
 - ☐ Low
 - ☒ **High**
 - ☐ Variable
22. *Functional programming minimizes _____ in computation.*
- ☐ Variables
 - ☐ Loops
 - ☒ **Side effects**
 - ☐ Functions
23. *Which model translates entire code before execution?*
- ☒ **Compiled**
 - ☐ Interpreted
 - ☐ Hybrid
 - ☐ Scripted
24. *Compiler design impacts efficiency of execution.*
- ☐ Sometimes
 - ☐ False
 - ☒ **True**
 - ☐ Rarely

25. *Interpreted languages provide easier _____.*
- ☐ Compilation
 - ☒ **Debugging**
 - ☐ Linking
 - ☐ Optimization
26. *Bytecode is executed by a _____.*
- ☐ Linker
 - ☐ Compiler
 - ☐ Assembler
 - ☒ **Virtual machine**
27. *Which language uses bytecode and virtual machine execution?*
- ☒ **Java**
 - ☐ C
 - ☐ Assembly
 - ☐ HTML
28. *Which paradigm is most suitable for symbolic reasoning and rules?*
- ☐ Functional
 - ☐ Procedural
 - ☒ **Logic**
 - ☐ Imperative
29. *Semantics ensures correct interpretation of programs.*
- ☐ False
 - ☒ **True**
 - ☐ Sometimes
 - ☐ Rarely
30. *Which phase checks grammatical structure?*
- ☐ Execution
 - ☐ Lexical analysis
 - ☒ **Parsing**
 - ☐ Linking
31. *Which model requires a compiler tool chain?*
- ☒ **Compiled**
 - ☐ Interpreted
 - ☐ Hybrid
 - ☐ Dynamic
32. *Pragmatics relates to _____ usage.*
- ☒ **Practical**
 - ☐ Structural
 - ☐ Formal
 - ☐ Logical

33. Which phase improves performance without changing output?
- ☐ Loading
 - ☐ Parsing
 - ☐ Scanning
 - ☒ Optimization
34. A token is a _____ of characters with meaning.
- ☐ single character
 - ☒ group
 - ☐ syntax tree
 - ☐ instruction
35. In `a = b + c;` what type of token is `=`?
- ☐ Identifier
 - ☒ Operator
 - ☐ Literal
 - ☐ Keyword
36. In a parse tree, what do the leaf nodes represent?
- ☐ Production rules
 - ☐ The start symbol
 - ☐ Non-terminal symbols
 - ☒ Terminal symbols
37. Why is maintainability higher in modern parser generators?
- ☒ Direct grammar-to-code mapping
 - ☐ More lines of code
 - ☐ No error messages
 - ☐ Uses older standards
38. What is the primary focus of Semantics in the context of programming languages?
- ☐ Code translation
 - ☒ Program meaning
 - ☐ Code structure
 - ☐ Execution speed
39. In EBNF, which notation is used to represent an element that can appear zero or more times?
- ☐ `< >`
 - ☒ `{ }`
 - ☐ `( )`
 - ☐ `[ ]`
40. In the code `a = b + c;` what does `a` represent to the lexical analyzer?
- ☒ An entry in the symbol table with the token id
 - ☐ A constant value
 - ☐ A syntax error
 - ☐ A special symbol

41. *Listener and Visitor patterns are used to separate _____ from application logic in modern parsing tools.*
- ☐ machine code
 - **grammar definitions**
 - ☐ memory management
 - ☐ token streams
42. *In Backus-Naur Form (BNF), what does the symbol `::=` represent?*
- ☐ Is equal to
 - ☐ Is not defined as
 - **May expand into**
 - ☐ Must terminate with
43. *Which semantics focuses on describing the sequence of computational steps to execute a program?*
- ☐ Axiomatic Semantics
 - ☐ Denotational Semantics
 - ☐ Static Semantics
 - **Operational Semantics**
44. *The focus of Denotational Semantics is on:*
- ☐ Transitions
 - ☐ Interpreting instructions
 - ☐ Step-by-step traces
 - **Mapping to a domain**
45. *What is the relationship between BNF and Context-Free Grammar (CFG)?*
- **BNF is a notation used to express context-free grammars**
 - ☐ BNF is used for hardware design while CFG is for software
 - ☐ BNF is a successor that replaced CFG entirely
 - ☐ There is no relationship between the two
46. *Which type of operational semantics traces exact execution steps, such as `x := 3+2` becoming `x := 5`?*
- ☐ Fixed-point semantics
 - ☐ Big-step semantics
 - ☐ Natural semantics
 - **Small-step semantics**
47. *ANTLR v4 uses _____ with dynamic lookahead for parsing.*
- ☐ LL(k)
 - **ALL(*)**
 - ☐ LR(1)
 - ☐ PEG
48. *In BNF, nonterminals are written using _____, while terminals are not.*
- ☐ square brackets []
 - **angle brackets < >**
 - ☐ curly braces { }
 - ☐ parentheses ()

49. What is the primary purpose of Data-Driven Design (DDD)?

- ☐ Automate code writing
- ☐ Use intuition for design
- ☐ Create designs without metrics
- **Use data to guide decisions**

50. Why was BNF created?

- **Precise, unambiguous formal specification**
- ☐ Easier human-readable code
- ☐ Faster program execution
- ☐ Replace binary with English

51. Which parsing tool is known for using the Earley algorithm to parse any context-free grammar efficiently?

- ☐ PEG.js
- **Nearley.js**
- ☐ Parsimmon
- ☐ ANTLR

52. ANTLR v4 is capable of generating parsers in multiple languages. Which list correctly reflects this?

- ☐ Only Java and C++
- ☐ Binary machine code only
- ☐ HTML and CSS only
- **Java, C#, and Python**

53. Which phase ensures logical correctness and detects type errors?

- **Semantic Analysis**
- ☐ Lexical Analysis
- ☐ Intermediate Code Generation
- ☐ Syntax Analysis

54. What is the main role of lexical analysis in a compiler?

- **Convert source code into tokens**
- ☐ Generate machine code
- ☐ Create a syntax tree
- ☐ Verify semantic correctness

55. In which compiler phase are parse trees used to check grammar?

- **Syntax analysis**
- ☐ Lexical analysis
- ☐ Semantic analysis
- ☐ Code optimization

56. Defining property of PEGs?

- ☐ Ambiguous
- ☐ Only regular languages
- ☐ Use NFA
- **Deterministic & unambiguous**

57. *What does SOS stand for in the context of operational semantics?*

- **Structural Operational Semantics**
- Sequential Operational Systems
- Static Object Semantics
- Standard Operational Sequences

58. *Compilers ensure that code is correct both syntactically and semantically before execution.*

- **True**
- False

59. *Data-driven design uses _____ data to improve grammar rules.*

- Random
- **Real-world**
- Theoretical
- Simulated

60. *In a parse tree for the expression $8 - 4 / 2$, which operator is evaluated first?*

- -, because it appears first
- **/, because it has higher precedence**
- Both simultaneously
- The leftmost operator

61. *Compared to older tools like Yacc, what is a key benefit of modern parser generators?*

- Lower memory usage
- Strict adherence to LALR parsing
- Manual hand-writing of all state machines
- **Superior error reporting and recovery**

62. *In data-driven design, _____ testing is used to compare two design variations.*

- Backus-Naur Mapping
- **A/B Testing**
- Integer string evolution
- Generative Software Design

63. *What is checked during semantic analysis using the parse tree?*

- **Type checking and scope resolution**
- Machine code generation
- Token creation
- Instruction scheduling

64. *Consider the BNF rule: ::= |*

- **Recursion**
- Terminal definition
- Iteration
- Optionality

65. *In Extended BNF (EBNF), what is the purpose of square brackets []?*
- ☐ To indicate a required sequence
 - **To indicate that an expansion is optional**
 - ☐ To indicate repetition one or more times
 - ☐ To define a character class
66. *The modern successor to PEG.js for generating compact parsers in JavaScript is _____.*
- ☐ Nearley
 - **Peggy**
 - ☐ Myna
 - ☐ Parsimmon
67. *Which of the following is true about Rust's memory management?*
- ☐ It uses garbage collection
 - ☐ It uses reference counting
 - **It uses ownership and borrowing**
 - ☐ It uses manual memory management
68. *What is the main advantage of dynamic typing?*
- **Flexibility in variable usage**
 - ☐ Early error detection
 - ☐ Compiler optimization
 - ☐ Memory safety
69. *What benefit does dynamic typing provide?*
- ☐ Faster compilation
 - ☐ Type safety at compile-time
 - **Code flexibility**
 - ☐ Strict memory management
70. *What does Gradual typing reduce?*
- ☐ Need for runtime type checks entirely
 - ☐ Compilation errors completely
 - ☐ Memory usage drastically
 - **Type-related runtime errors partially**
71. *What happens in weakly typed languages when adding a string and a number?*
- ☐ Compile-time error
 - ☐ Runtime error
 - **Automatic type coercion**
 - ☐ Program halts
72. *Bounded generics restrict _____.*
- **What types can be passed**
 - ☐ Memory usage
 - ☐ Function overloading
 - ☐ Runtime type errors

73. **Which combination helps ensure fewer runtime errors?**
- ☐ Dynamic typing
 - ☐ Weak typing
 - ☒ **Strong typing + null safety**
 - ☐ Optional typing
74. **Gradual typing eliminates all runtime type errors.**
- ☐ True
 - ☒ **False**
75. **What does borrowing allow in Rust?**
- ☐ Transferring ownership of a value
 - ☐ Duplicating a value
 - ☒ **Creating references to a value without taking ownership**
 - ☐ Ignoring the value
76. **Which polymorphism resolves at runtime?**
- ☐ Parametric
 - ☐ Ad-hoc
 - ☒ **Subtype**
 - ☐ Generic
77. **What does Gradual typing allow?**
- ☒ **Mixing static and dynamic types in a program**
 - ☐ Only dynamic typing
 - ☐ Only static typing
 - ☐ Ignoring types completely
78. **What does hybrid typing aim to achieve?**
- ☒ **Combine the benefits of static and dynamic typing**
 - ☐ Replace dynamic typing completely
 - ☐ Eliminate compile-time checks
 - ☐ Only support weak typing
79. **Type inference works best with _____.**
- ☐ Weakly typed languages
 - ☒ **Strongly typed languages**
 - ☐ Unstructured programming languages
 - ☐ Dynamic languages only
80. **Which feature allows early detection of type errors in Java?**
- ☐ Dynamic typing
 - ☐ Strong typing
 - ☒ **Static typing**
 - ☐ Null safety
81. **Which combination gives maximum runtime flexibility?**
- ☐ Static + strong
 - ☐ Dynamic + strong
 - ☐ Static + weak
 - ☒ **Dynamic + weak**

82. *How does Gradual typing help in large codebases?*
- **It allows incremental adoption of static typing**
 - It removes the need for types entirely
 - It converts all types to dynamic automatically
 - It avoids all runtime checks
83. *What is the primary purpose of Rust's ownership system?*
- To manage memory through garbage collection
 - **To ensure memory safety without a garbage collector**
 - To simplify syntax
 - To enable dynamic typing
84. *In TypeScript, any type represents _____.*
- A fixed type
 - Ownership rules
 - **Fully dynamic typing**
 - Null safety
85. *Queue ADT follows which order?*
- LIFO
 - **FIFO**
 - Random
 - Stack order
86. *Which programming feature supports abstraction?*
- **Classes and objects**
 - Pointers
 - Memory allocation
 - Loops
87. *Which typing discipline allows early detection of type errors?*
- Dynamic typing
 - **Static typing**
 - Weak typing
 - Null safety
88. *Weakly typed languages may allow adding a string and a number without error.*
- **True**
 - False
89. *What are getter and setter methods used for?*
- Abstraction
 - Inheritance
 - **Encapsulation**
 - Polymorphism
90. *Which of these is NOT a type of polymorphism?*
- Ad-hoc polymorphism
 - Parametric polymorphism
 - Subtype polymorphism
 - **Sequential polymorphism**

91. *What is the main purpose of generics?*
- ☐ To reduce memory footprint
 - ☐ To increase performance
 - **To write type-safe reusable code**
 - ☐ To replace inheritance
92. *A Stack ADT follows which order?*
- ☐ FIFO
 - **LIFO**
 - ☐ Random
 - ☐ Priority
93. *Which language is statically and strongly typed?*
- ☐ JavaScript
 - ☐ Python
 - ☐ C
 - **Java**
94. *What benefit does static typing provide?*
- ☐ Flexibility
 - **Early error detection**
 - ☐ Easier scripting
 - ☐ Late binding
95. *Gradual typing allows migrating _____.*
- ☐ From statically typed code to dynamic code only
 - **From untyped code to fully typed code incrementally**
 - ☐ From weak typing to strong typing automatically
 - ☐ Only within one function
96. *Combining generics and polymorphism increases code safety and reusability.*
- **True**
 - ☐ False
97. *Which is NOT a benefit of type inference?*
- ☐ Reduces boilerplate
 - ☐ Enables safer refactoring
 - **Slows down compilation**
 - ☐ Catches type errors at compile time
98. *Ownership and borrowing improve both memory safety and concurrency safety.*
- **True**
 - ☐ False
99. *Dynamic typing prevents runtime errors completely.*
- ☐ True
 - **False**